

Lecture 2

Fundamentals of Numerical Analysis

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

What we'll do:

- get introduced to the basics of Matlab
- learn how to *test* numerical methods, not just run them
- errors and Taylor series

Why this is important:

- When developing code often a proof-of-concept is easier in a higher-level environment
 - ▶ Matlab or Julia (open source)
 - ▶ Python + Numpy + Scipy front-end, C/C++ backend
 - ▶ R (statistical)
 - ▶ ...
- When posed with a problem, we have a myriad of options (a.k.a. methods) . To choose the correct tools, we need to know how well it “works”
 - ▶ accuracy for the problem
 - ▶ speed for the problem
 - ▶ implementation ease
 - ▶ memory footprint
- Our testing and our methods often rely on “randomness” and the impact of finite precision
 - ▶ Random input
 - ▶ Random perturbations
 - ▶ Random sequences

What is MATLAB?

- both a computing environment and a language
- initially developed as an easy interface to LAPACK: Linear Algebra Package (FORTRAN libraries)
- **MAT**rix **LAB**oratory
- Written in C. For matrix computations, it calls C/Fortran libraries == Fast
- Matlab + "Toolboxes"
 - Symbolic Math Toolbox: mathematical manipulation of symbols
 - Partial Differential Equation Toolbox: tools for solving PDEs in 2-D
 - Statistics Toolbox: statistical data analysis
 - Image processing toolbox: visualization and image analysis
 - Bioinformatics toolbox: computational molecular biology
 - Compiler: application development
 - many many more.
- <http://www.mathworks.com>

Why Matlab?

The Good:

- Fast development times
- no compiling, easy debugging
- accessible syntax and language constructs
- in-house graphics capabilities
- tons of basic "libraries" or functions available
- many more complicated "toolboxes" can be added

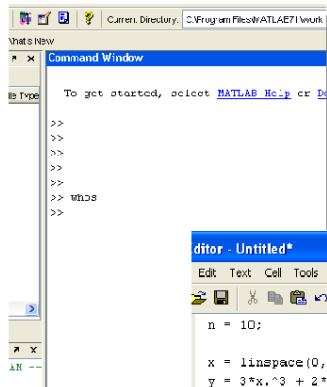
The Bad:

- small coding mistakes can result in slow code
- loops are extremely computationally intensive
- language is limited: no templates, classes etc.

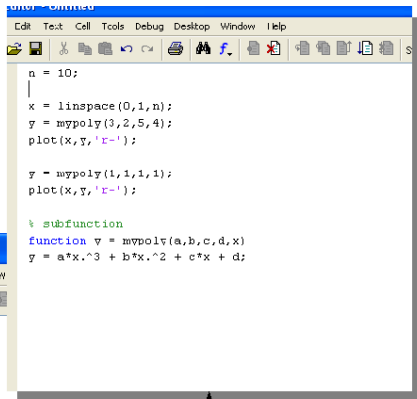
The Ugly:

- proprietary (but the language format is open)
- expensive (but free for USU students/faculty through webstore)
- the open source substitute, Octave, is not fully compatible

Interpreter vs. Scripts vs. Functions



One copy of x



Copy of x for each function call

Interpreter vs. Scripts vs. Function

Interpreter:

- quick access
- good for developing ideas
- good for accessing data in the workspace
- poor performance and editing/debugging capabilities

Script

- good test bed: all info in the workspace
- fast, linear
- poor profiling/optimization

Functions

- can be optimized and cached
- don't fill up the workspace with memory that you have to manage, but...
- pass by value (ouch!)
- reusable code

Functions

- save in *functionname.m*
- can have 0, n, or variable number of inputs
- can have 0, n, or variable number of outputs
- use `return;` to leave a function early
- use `inline(...)` to define without another file
- variables not "passed in" are *local*: only available within the function scope
- use `global varname;` to declare a variable that is visible through all functions

More on Matlab...

- Free numerics book by Cleve Moler:
<http://www.mathworks.com/moler/index.html>
- Extensive documentation and tutorials:
<https://www.mathworks.com/help/matlab/>
- Matlab Online (browser version, no installation):
<https://matlab.mathworks.com>
- Online AI assistants (e.g., ChatGPT) can help explain Matlab syntax, debugging, and workflow tips

Last Time: Measuring Error

- Here we used the absolute error:

$$\begin{aligned}\text{Error}(x_a) &= |x_t - x_a| \\ &= |\text{true value} - \text{approximate value}|\end{aligned}$$

- This doesn't tell the whole story. For example, if the values are large, like billions, then an Error of 100 is small. If the values are smaller, say around 10, then an Error of 100 is large. We need the relative error:

$$\begin{aligned}\text{Rel}(x_a) &= \left| \frac{x_t - x_a}{x_t} \right| \\ &= \left| \frac{\text{true value} - \text{approximate value}}{\text{true value}} \right|\end{aligned}$$

- More on *types* of errors next time...

Taylor

- All we can ever do is add and multiply (more on the FPU later)
- We can't directly evaluate e^x , $\cos(x)$, $\sqrt{x}$
- What to do? Taylor Series *approximation*

Taylor

The Taylor series expansion of $f(x)$ at the point $x = c$ is given by

$$\begin{aligned} f(x) &= f(c) + (x - c)f'(c) + \frac{(x - c)^2}{2!}f''(c) + \cdots + \frac{(x - c)^n}{n!}f^{(n)}(c) + \cdots \\ &= \sum_{k=0}^{\infty} \frac{(x - c)^k}{k!}f^{(k)}(c) \end{aligned}$$

Taylor Example

Taylor

The Taylor series expansion of $f(x)$ about the point $x = c$ is given by

$$\begin{aligned} f(x) &= f(c) + (x - c)f'(c) + \frac{(x - c)^2}{2!}f''(c) + \cdots + \frac{(x - c)^n}{n!}f^{(n)}(c) + \cdots \\ &= \sum_{k=0}^{\infty} \frac{(x - c)^k}{k!}f^{(k)}(c) \end{aligned}$$

Example (e^x)

We know $e^0 = 1$, so expand about $c = 0$ to get

$$\begin{aligned} f(x) &= e^x = 1 + (x - 0) \cdot 1 + \frac{(x - 0)^2}{2} \cdot 1 + \cdots \\ &= 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots \end{aligned}$$

Taylor Approximation

- So

$$e^2 = 1 + 2 + \frac{2^2}{2!} + \frac{2^3}{3!} + \dots$$

- But we can't evaluate an infinite series, so we truncate...

Taylor Series Polynomial Approximation

The Taylor Polynomial of degree n for the function $f(x)$ about the point c is

$$p_n(x) = \sum_{k=0}^n \frac{(x-c)^k}{k!} f^{(k)}(c)$$

Example (e^x)

In the case of the exponential

$$e^x \approx p_n(x) = 1 + x + \frac{x^2}{2!} + \dots + \frac{x^n}{n!}$$

Taylor Approximation

Evaluate e^2 :

- Using 0th order Taylor series: $e^x \approx 1$ does not give a good fit.
- Using 1st order Taylor series: $e^x \approx 1 + x$ gives a better fit.
- Using 2nd order Taylor series: $e^x \approx 1 + x + x^2/2$ gives a really good fit.

```
1 x=2;  
2 pn=0;  
3 for j=0:15  
4     pn = pn + (x^j)/factorial(j);  
5     err = exp(2)-pn  
6     pause  
7 end
```

Taylor Approximation Recap

Infinite Taylor Series Expansion (exact)

$$f(x) = f(c) + (x - c)f'(c) + \frac{(x - c)^2}{2!}f''(c) + \cdots + \frac{(x - c)^n}{n!}f^{(n)}(c) + \cdots$$

Finite Taylor Series Expansion (exact)

$$f(x) = f(c) + (x - c)f'(c) + \frac{(x - c)^2}{2!}f''(c) + \cdots + \frac{(x - c)^n}{n!}f^{(n)}(\xi),$$

but we don't know ξ .

Finite Taylor Series Approximation

$$f(x) \approx f(c) + (x - c)f'(c) + \frac{(x - c)^2}{2!}f''(c) + \cdots + \frac{(x - c)^n}{n!}f^{(n)}(x),$$

Taylor Approximation Error

- How accurate is the Taylor series polynomial approximation?
- The n terms of the approximation are simply the first n terms of the *exact* expansion:

$$e^x = \underbrace{1 + x + \frac{x^2}{2!}}_{p_2 \text{ approximation to } e^x} + \underbrace{\frac{x^3}{3!} + \dots}_{\text{truncation error}} \quad (1)$$

- So the function $f(x)$ can be written as the Taylor Series approximation plus an error (truncation) term:

$$f(x) = f_n(x) + E_n(x)$$

where

$$E_n(x) = \frac{(x - c)^{n+1}}{(n + 1)!} f^{(n+1)}(\xi)$$

Big "O" (different view)

We write the error as

$$\begin{aligned} E_n(x) &= \frac{(x-c)^{n+1}}{(n+1)!} f^{(n+1)}(\xi) \\ &= \mathcal{O}\left(\frac{(x-c)^{n+1}}{(n+1)!}\right) \end{aligned}$$

since we assume the $(n+1)^{th}$ derivative is bounded on the interval $[a, b]$.

Often, we let $h = x - c$ and we have

$$f(x) = p_n(x) + \mathcal{O}(h^{n+1})$$

Truncation Error

The Taylor series expansion of $\sin(x)$ is

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} - \dots$$

If $x \ll 1$, then the remaining terms are small. If we neglect these terms

$$\sin(x) = \underbrace{x - \frac{x^3}{3!} + \frac{x^5}{5!}}_{\text{approximation to sin}} \underbrace{- \frac{x^7}{7!} + \frac{x^9}{9!} - \dots}_{\text{truncation error}}$$

Taylor Series

- How do we evaluate $f(x) = \frac{1}{1-x}$ computationally?
- Taylor Series Expansion:

$$f(x) = f(c) + (x - c)f'(c) + \frac{(x - c)^2}{2!}f''(c) + \cdots + \frac{(x - c)^n}{n!}f^{(n)}(\xi),$$

- Thus with $c = 0$

$$\frac{1}{1-x} = 1 + x + x^2 + x^3 + \cdots$$

- Second order approximation:

$$\frac{1}{1-x} \approx 1 + x + x^2$$

Taylor Errors

- How many terms do I need to make sure my error is less than 2×10^{-8} for $x = 1/2$?

$$\frac{1}{1-x} = 1 + x + x^2 + \cdots + x^n + \sum_{k=n+1}^{\infty} x^k$$

- so the error at $x = 1/2$ is

$$\begin{aligned} e_{x=1/2} &= \sum_{k=n+1}^{\infty} \left(\frac{1}{2}\right)^k = \frac{(1/2)^{n+1}}{1-1/2} \\ &= 2 \cdot (1/2)^{n+1} < 2 \times 10^{-8} \end{aligned}$$

- then we need

$$\begin{aligned} n+1 &> \frac{-8}{\log_{10}(1/2)} \approx 26.6 \quad \text{or} \\ n &> 26 \end{aligned}$$

Errors

- How do I classify my method?
- Goal: determine how the error $|f(x) - p_n(x)|$ behaves relative to n (and f).
- Goal: determine how the cost of computing $p_n(x)$ behave relative to n (and f).
- for $f(x) = \frac{1}{1-x}$ we have

$$p_n = \sum_{k=0}^n x^k = 1 + x + x^2 + \dots$$

- so

$$e_n = |f(x) - p_n(x)|$$

- Is $e_n \sim 1/n^r$?
- Is $e_n \sim 1/\sqrt{n}$?
- Is $e_n \sim 1/n!$?

Big-O

How to measure the impact of n on algorithmic cost?

$\mathcal{O}(\cdot)$

Let $g(n)$ be a function of n . Then define

$$\mathcal{O}(g(n)) = \{f(n) \mid \exists c, n_0 > 0 : 0 \leq f(n) \leq cg(n), \forall n \geq n_0\}$$

That is, $f(n) \in \mathcal{O}(g(n))$ if there is a constant c such that $0 \leq f(n) \leq cg(n)$ is satisfied.

- assume non-negative functions (otherwise add $|\cdot|$) to the definitions
- $f(n) \in \mathcal{O}(g(n))$ represents an asymptotic upper bound on $f(n)$ up to a constant
- example: $f(n) = 3\sqrt{n} + 2\log n + 8n + 85n^2 \in \mathcal{O}(n^2)$

Big-O (Omicron)

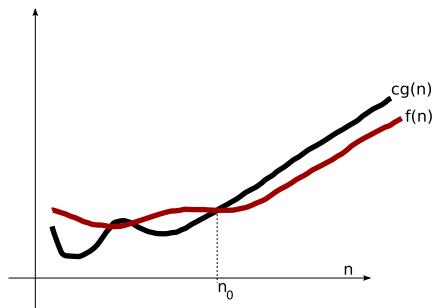
asymptotic upper bound

$\mathcal{O}(\cdot)$

Let $g(n)$ be a function of n . Then define

$$\mathcal{O}(g(n)) = \{f(n) \mid \exists c, n_0 > 0 : 0 \leq f(n) \leq cg(n), \forall n \geq n_0\}$$

That is, $f(n) \in \mathcal{O}(g(n))$ if there is a constant c such that $0 \leq f(n) \leq cg(n)$ is satisfied.



Big-Omega

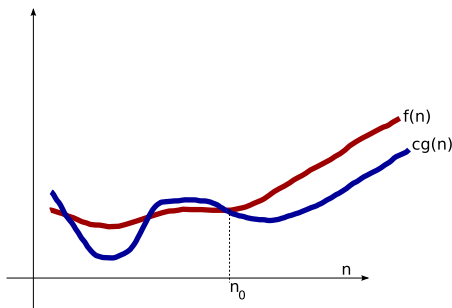
asymptotic lower bound

$\Omega(\cdot)$

Let $g(n)$ be a function of n . Then define

$$\Omega(g(n)) = \{f(n) \mid \exists c, n_0 > 0 : 0 \leq cg(n) \leq f(n), \forall n \geq n_0\}$$

That is, $f(n) \in \Omega(g(n))$ if there is a constant c such that $0 \leq cg(n) \leq f(n)$ is satisfied.



Big-Theta

asymptotic tight bound

$\Theta(\cdot)$

Let $g(n)$ be a function of n . Then define

$$\Theta(g(n)) = \{f(n) \mid \exists c_1, c_2, n_0 > 0 : 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n), \forall n \geq n_0\}$$

Equivalently, $\Theta(g(n)) = \mathcal{O}(g(n)) \cap \Omega(g(n))$.

